

Task 5: Containerize devops-crm-project with Docker

1. Objective

Containerize the Twenty CRM app (devops-crm-project) using Docker, so it can be built and run consistently with docker compose up, without requiring yarn/node set up directly on the host machine.

2. Understanding the project's architecture first

Before writing any Docker files, I researched how a Twenty app actually runs, since this project is not a typical standalone web server.

Key finding: this app doesn't serve its own web UI on its own port. It's a set of code definitions (page layouts, entities, front components) that get built and **synced into a separate, full Twenty server instance**. That server — normally started locally via yarn twenty docker:start — is itself a Docker image (twentycrm/twenty-app-dev) that provides the actual UI, GraphQL API, and database, on port 2020.

This meant the containerization needed **two services working together**, not one:

1. **server** — the Twenty server itself (twentycrm/twenty-app-dev image), which the app syncs into.
2. **app** — a container built from this project's own code, which installs dependencies and runs yarn twenty dev to build and sync against the server service.

3. Dockerfile — explained

dockerfile

---- Stage 1: deps ----

FROM node:24-alpine AS deps

WORKDIR /app

RUN corepack enable

COPY package.json yarn.lock .yarnrc.yml ./

RUN yarn install --immutable

---- Stage 2: runtime ----

FROM node:24-alpine AS runtime

WORKDIR /app

RUN corepack enable

RUN addgroup -S appgroup && adduser -S appuser -G appgroup

COPY --from=deps /app/node_modules ./node_modules

COPY --from=deps /app/package.json /app/yarn.lock /app/yarnrc.yml ./

COPY . .

RUN chown -R appuser:appgroup /app

USER appuser

CMD ["yarn", "twenty", "dev"]

Design decisions:

- **Multi-stage build.** Stage 1 (deps) installs dependencies in its own layer. Docker caches this layer and only re-runs yarn install when package.json/yarn.lock actually change — not on every source edit — which speeds up rebuilds significantly.
- **node:24-alpine** — matches the Node version pinned in .nvmrc (24.5.0), and Alpine keeps the base image small.
- **Non-root user.** A dedicated appuser/appgroup is created, and the final USER appuser line means the app never runs as root inside the container — a core Docker security best practice.
- **No EXPOSE.** This container doesn't listen on its own port — it's a CLI process syncing to the server service — so no port is exposed here (the server service is what's exposed, in docker-compose.yml).
- **CMD ["yarn", "twenty", "dev"]** — runs the same command used manually during local development, so the container's behavior matches what "running the app" means for this project.

4. .dockerignore — explained

node_modules/

.yarn/

!.yarn/releases

!.yarn/patches

!.yarn/plugins

.git/

.github/
dist/
build/
.twenty/
coverage/
.env
.env.*
.vscode/
.idea/
.DS_Store
*.md
*.pdf

Why each group matters:

- node_modules/ — installed fresh inside the image; copying the host's copy in would bloat the build context and can cause platform mismatches (e.g. native modules built for Windows breaking inside Linux).
- .git/, .github/ — version control metadata has no place inside a runtime image.
- dist/, build/, .twenty/, coverage/ — generated/local artifacts, rebuilt fresh inside the container.
- .env* — never bake secrets or environment-specific config into an image; these should be supplied at runtime instead.
- Editor/OS files and documentation — irrelevant to the running app, excluding them keeps the build context (and therefore build time) small.

Issue faced: the file was initially created without its leading dot (dockerignore instead of .dockerignore), so Docker didn't recognize it at all. Fixed with mv dockerignore .dockerignore.

5. docker-compose.yml — explained

yaml

services:

server:

image: twentycrm/twenty-app-dev:latest

container_name: twenty-app-dev-server

ports:

- "2020:2020"

volumes:

- twenty-app-dev-data:/app/data

- twenty-app-dev-storage:/app/storage

restart: unless-stopped

healthcheck:

test: ["CMD", "wget", "-qO-", "http://localhost:2020/healthz"]

interval: 10s

timeout: 5s

retries: 10

app:

build:

context: .

dockerfile: Dockerfile

container_name: devops-crm-app

depends_on:

server:

condition: service_healthy

environment:

- TWENTY_SERVER_URL=http://server:2020

volumes:

- ./src:/app/src

restart: unless-stopped

volumes:

twenty-app-dev-data:

twenty-app-dev-storage:

Design decisions:

- **Two services** — server (the Twenty platform) and app (this project's own definitions), matching the real architecture described in section 2.
- **Port mapping 2020:2020** — confirmed correct by testing: after docker compose up, docker ps showed 0.0.0.0:2020->2020/tcp and the server reported (healthy), and http://localhost:2020 loaded the Twenty login page.
- **Named volumes** (twenty-app-dev-data, twenty-app-dev-storage) — persist the server's database and file storage across container restarts, so you don't lose your workspace data every time you run docker compose down / up again.
- **depends_on with condition: service_healthy** — the app service won't start until the server service's healthcheck passes, avoiding a race condition where the app tries to sync before the server is ready.
- **environment: TWENTY_SERVER_URL** — points the app container at the server by its Compose network name (server) rather than localhost, since each service runs in its own container with its own network namespace.
- **Source volume mount (./src:/app/src)** — allows editing source on the host and having changes reflected without rebuilding the image, useful during active development.

6. Code fix included in this branch

While testing, yarn twenty dev:build/dev failed with:

```
INVALID_PAGE_LAYOUT_WIDGET_DATA: Position layoutMode "GRID" does not match  
tab layoutMode "VERTICAL_LIST"
```

This is a pre-existing bug in src/page-layouts/main-page.page-layout.ts — the tab was set to VERTICAL_LIST while its widget used GRID-only positioning (gridPosition). Fixed by changing the tab's layoutMode to GRID to match the widget. Without this fix, the app container would fail to sync inside Docker the same way it failed in CI.

7. How to build, run, and stop

Build the images:

```
bash
```

```
docker compose build
```

Start everything (foreground, see live logs):

bash

docker compose up

Start everything in the background:

bash

docker compose up -d

Check status of both containers:

bash

docker compose ps

View logs for a specific service:

bash

docker compose logs server

docker compose logs app

Stop everything (containers stopped, data volumes preserved):

bash

docker compose down

Stop and also remove volumes (full reset, wipes workspace data):

bash

docker compose down -v

8. How this was tested

1. docker compose build — confirmed both stages of the Dockerfile build without error.
2. docker compose up — started both services, watched logs to confirm the server completed its startup migrations.
3. docker compose ps / docker ps — confirmed the server container showed Up (healthy) and the port mapping 0.0.0.0:2020->2020/tcp was actually applied.
4. curl http://localhost:2020 and opening the URL in a browser — confirmed the Twenty login page actually loads.

5. docker compose logs app — checked the app container's own logs to confirm it built/synced against the server service.

9. Issues faced and solutions

Issue	Cause	Solution
.dockerignore not being respected	File was named dockerignore, missing the leading dot	Renamed with mv dockerignore .dockerignore
localhost:2020 not loading initially	Server container was still finishing startup (running database migrations) when first checked	Waited for the healthcheck to report (healthy) before testing the URL again
App's build/sync failing	Pre-existing INVALID_PAGE_LAYOUT_WIDGET_DATA bug in main-page.page-layout.ts (same bug found during the CI task)	Fixed the tab's layoutMode to GRID, included in this branch's commit
Uncertainty about server's real port/volumes before testing	Twenty's public docs describe the twenty-app-dev image and its volumes at a high level, but exact internal paths aren't documented	Verified empirically via docker ps (port) after running the stack; volume paths are a reasonable best-effort based on standard Twenty server conventions — confirmed the stack runs correctly end-to-end

10. Git / PR workflow

```
bash
```

```
git clone https://github.com/PearlThoughts-Intern-DevOps/devops-crm-project.git
devops-crm-project-task5
```

```
cd devops-crm-project-task5
```

```
git checkout main
```

```
git pull origin main
```

```
git checkout -b fiza-task5
```

```
# add Dockerfile, .dockerignore, docker-compose.yml
```

```
# fix layoutMode bug in src/page-layouts/main-page.page-layout.ts
```

```
git add Dockerfile .dockerignore docker-compose.yml src/page-layouts/main-  
page.page-layout.ts
```

```
git commit -m "Add Docker setup (Dockerfile, docker-compose, .dockerignore) and fix  
layoutMode bug"
```

```
git push -u origin fiza-task5
```

PR opened: fiza-task5 → main, in PearlThoughts-Intern-DevOps/devops-crm-project.